Microcode Paper — Draft Sections 3 & 4 (preview)

Contents

1 Microcode as an Implementational Layer

Prior work has studied microcode as an object to be reverse engineered and, latterly, as a surface on which to mount security mechanisms. We instead treat it as a programming target: a layer beneath the instruction set on which a cryptographic primitive can be expressed directly, and on which it can be made to run faster than any code the instruction set itself admits. This section introduces the layer in the terms a reader will need, namely what a triad is, what the writable patch RAM is, and by what mechanism a stock instruction is customised to run code of our choosing, and it then sets out the methodology that turns those ingredients into a verified patch on the Intel Goldmont Celeron N3350. The same methodology underlies every primitive in this work, namely the field arithmetic for P-256, Curve25519, secp256k1 and Poly1305 together with the Keccak-f[1600] permutation of Section 2, and we therefore present it once, in general terms, and instantiate it twice in the sections that follow.

Two properties of the layer motivate the entire effort, and both will recur as the source of every result we report. The first is that the micro-architectural integer register file is twice the size of the architectural one, comprising 16 general-purpose and 16 temporary registers, which lets a primitive

keep far more state resident than the roughly 14 usable general-purpose registers of native x86 and thereby removes the register spills that dominate compiled code on a register-starved in-order core; this is the property that Section 2 exploits to hold an entire Keccak state in registers across all 24 rounds. The second is that a triad issues up to three micro-ops as a single sequential bundle, which yields a denser and more hazard-aware packing than a linear instruction stream affords, and in particular lets several carry chains advance at once; this is the property that Section 3 exploits to overlap the multiply–accumulate–carry chain of a reduced field multiplication. The price of admission is a hard budget of 128 triads, together with a body of hardware behaviour that is nowhere documented and that we were obliged to map empirically.

1.1 Microcode, Triads, and the Patch RAM

From macroinstructions to micro-ops. The x86 instruction set is a contract between software and the hardware that runs it, a stable interface that fixes what each instruction must appear to do while saying nothing about how the hardware is to achieve it, and it is not the language the core actually executes. A decode unit translates each architectural instruction, which we call a *macroinstruction*, into one or more internal *micro-ops* (μops) drawn from a simpler, RISC-like repertoire that the execution units actually run. Frequent and simple macroinstructions are handled by fast hardware decoders that emit a handful of μops directly, whereas complex or rare ones are instead decoded by a microcode sequencer. It is for these latter, microcoded instructions that the μop program implementing the macroinstruction is held on an on-chip microcode ROM, the MSROM, from which the sequencer fetches it. Goldmont exposes more than two thousand distinct μops, and a single complex macroinstruction may expand to dozens of them. The encoding is *vertical*, meaning that each μop resembles a compact RISC instruction with an operation field and a small number of operand fields rather than a wide horizontal control word. The reverse engineering of Koppe et al. [**?**] on AMD K8/K10 first established this structure publicly, organising the μops into operation classes for register arithmetic, loads, stores and special operations, and Borrello et al. [**?**] subsequently recovered the corresponding μops for Goldmont together with their operand and immediate encodings.

The triad. Inside the microcode store the μops are not laid out singly but grouped into *triads*. A triad is a bundle of three μops together with one *sequence word*, and it is the unit in which microcode is addressed, fetched, and issued: the sequencer advances triad by triad, presenting the three μops of a triad to the execution units before it moves on. The sequence word is the control-flow field of the triad. It selects what the sequencer does after the triad's μops have issued, namely fall through to the next triad, branch to a named triad address, or signal that decoding of the macroinstruction is complete, and on Goldmont it can additionally impose a synchronisation barrier on the bundle, in the manner of an `lfence`. A triad is thus at once a packing unit and the granularity of control flow. It is a packing unit because its three μops are fetched and issued together as a single bundle, so that the quantity governing both the size of a program and the length of its dependency chains is the number of triads rather than the number of μops, and filling all three slots of every triad is in consequence the central concern of the packing described in Section 1.2. It is the granularity of control flow because the sequence word steers the sequencer one triad at a time and a branch can target only a triad boundary, never a μop within a triad, which has the practical consequence, developed in Section 1.4, that an operation setting a condition and the branch that tests it must be placed in the same triad. The MSROM on Goldmont holds on the order of eight thousand such triads, 7936 to be exact [**?**], which together form the μop program that decodes the entire x86 instruction set. Against that backdrop the 128 triads of writable patch RAM introduced below

amount to under two percent of the decode store, and it is within that small allowance that an entire cryptographic primitive must be expressed.

The encoding of a triad. The structure of a triad is worth making concrete, since it determines what a single μop can express and therefore what the packer of Section 1.4 is working against. We follow the Goldmont encoding first documented by Ermolov, Goryachy and Sklyarov's uCodeDisasm [**?**], whose μop mnemonics our toolchain inherits by way of lib-micro. Each μop is a 48-bit word. A 12-bit opcode names the operation, and for some operations also encodes a sub-mode such as a condition code or a data size; two 6-bit fields select the source registers and a third 6-bit field the destination, which for a memory operation instead supplies a second source. An operand need not be a register: when the top three bits of a source field are `001` that field's remaining bits and the μop's two immediate fields together form a 16-bit immediate, and when they are `010` the field indexes a read-only constants ROM. A few mode bits and a two-bit parity check complete the word. The 16-bit ceiling on an immediate is exactly why a 64-bit constant must be built by composition, a point Section 1.4 returns to. The sequence word is a narrower 30-bit word, but its fields act on the triad rather than on data. Three 2-bit pointers bind its actions to particular slots; a four-bit control-flow field selects among returning to the caller, looping within the microcode, stopping, and signalling that decoding is complete; a branch-address field supplies a jump target that becomes conditional when the preceding μop is a flag-setting test or subtract; and a three-bit synchronisation field can place a load fence or a full barrier on the bundle, which is the `lfence`-like behaviour noted above. These two fields together are the whole of control flow in the layer, and Section 1.4 records which of their forms behave reliably on this part.

The patch RAM. The MSROM is fixed at manufacture, but a CPU must be able to correct decode bugs after it ships. Vendors provide for this with a small writable microcode store, the MSRAM or *patch RAM*: in normal operation the firmware or operating system loads a cryptographically signed microcode patch into it at boot by writing the patch's address to a dedicated model-specific register, and the store is volatile and must be reloaded after every reset. It is one of several microcode-sequencer arrays, the others holding the read-only μops, their sequence words, and the match table described below. On Goldmont the patch RAM occupies the region `U7c00`–`U7e00` and holds at most 128 triads, and this ceiling, as Section 1.3 develops at length, is the binding resource of our entire approach. A patch is therefore a short array of triads, each consisting of three μops and a sequence word, written in exactly the format the MSROM itself uses.

Customising an instruction. A patch is inert until some macroinstruction is made to decode into it. This is the role of the *match-and-patch* mechanism, a table of 32 entries each of which redirects a read-only triad address to one in the patch RAM. When the sequencer is about to fetch the triad at a matched MSROM address it is instead sent to the corresponding triad in the MSRAM. Pointing a match entry at the MSROM entry point of a chosen macroinstruction therefore *hooks* that instruction, so that issuing it transfers control into our patch. Because each entry redirects an even-aligned pair of addresses at once, mapping both `src` and `src`+1 to consecutive patch triads, only even addresses can be hooked. This is the mechanism, and the only mechanism, by which we install code of our own: we write triads into the patch RAM and aim a match entry at an instruction we are willing to repurpose.

Three lines of prior work make this practical on Goldmont, and our toolchain rests on all of them. The foundation is due to Ermolov, Goryachy and Sklyarov: beginning in 2020 they obtained Intel's *Red Unlock* debug state on Goldmont through the Intel-SA-00086 Management

Engine vulnerability, identified the undocumented `udbgrd` and `udbgwr` instructions that read and write the microcode arrays from software [**?**], and released *uCodeDisasm*, the disassembler that first documented the Goldmont μop, triad and sequence-word encoding publicly [**?**]. Building on this, Borrello et al.'s CustomProcessingUnit (2023) reverse-engineered the patch and hook format, defined the semantics of some eight thousand μops, and provides a static decompiler together with an assembler from μop mnemonics to triads [**?**]. We build most directly on Krog's *lib-micro*, an eBPF-inspired C macro language for chaining μops into triads together with a C API that installs a patch and programs a match entry on an individual core, writing the sequencer arrays through the `udbgrd`/`udbgwr` interface rather than through the signed boot-time update path. Reaching that interface requires a part held in the Red-Unlock state above, which we reproduce on our Celeron N3350. Where this body of work used such access to study or to harden the CPU, we use it to make a cryptographic kernel the body of a hooked instruction, and the rest of the paper is concerned with doing so well. The two ways in which a primitive stresses the mechanism, control flow on the one hand and arithmetic density on the other, are the subjects of Section 2 and Section 3 respectively.

1.2 The Triad as an Issue Bundle and the Entry Path

The entry path. Every primitive shares a single entry path that instantiates the customisation mechanism of Section 1.1. We hook an otherwise unused macroinstruction, the `vmwrite` whose opcode is `0x0cd8`, by programming a match register so that its MSROM entry point is redirected into our patch in the MSRAM. The native side places the operand pointers in fixed architectural registers, with a base pointer in `RCX`, and issues a single `vmwrite`, whereupon control transfers into the patch. The patch reads its inputs, computes entirely in registers, writes its outputs through the base pointer, and returns when its final triad carries the sequence word that signals decode-complete. Because the hook is placed on a single static instruction, a complete operation executes within one firing, which, as Section 1.3 explains, is the property that makes register residency worthwhile. When two patches must be resident simultaneously we hook a distinct macroinstruction for each, since a single hooked instruction can be redirected to only one entry point; the Curve25519 field operations of Section 3, for example, fire the multiplication on `vmwrite` and the squaring on `vmread`, whose opcode is `0x0618`.

The triad as an issue bundle. Where Section 1.1 described the triad as the microcode store sees it, we now describe how we use it. We program each triad as a bundle of up to three μops together with its sequence word, and we rely throughout on a property that we verified rather than assumed, as reported in Section 1.4: the three slots execute with full sequential semantics over the register file. A value written in an earlier slot is visible to later slots, which is read-after-write forwarding; a read in an earlier slot observes the old value of a register that a later slot overwrites, which is the write-after-read case; and when two slots write the same register the later slot prevails, which is the write-after-write case. This holds identically for general-purpose and temporary registers, and for condition flags as well as for data. In practice the triad behaves like a small statically scheduled VLIW word with intra-bundle forwarding, so that a short dependent chain can be folded into a single issue group rather than spread across three instructions while independent work fills the remaining slots. The layer is most expressive precisely where compiled native code is weakest, namely in the dependent reductions and carry chains that the instruction set architecture would serialise; this is the lever behind both worked examples, where Section 2 folds a balanced parity tree into successive slots and Section 3 fuses a multiplication, an accumulation and a carry capture into one bundle.

Packing constraints. Two rules bound what may share a triad, and both are enforced by the hardware on pain of a hang or a silently incorrect result. The first is that at most one memory μop may occupy a triad, because a second load or store in the same triad hangs the core, which has only a single first-level-cache data port. The second is that a value produced by a load in an earlier slot is not forwarded to a later slot of the same triad, unlike a register result, so that a consumer of a freshly loaded value must begin a new triad. The asymmetry is worth stating explicitly, since arithmetic-to-arithmetic forwarding within a triad is permitted whereas load-to-arithmetic forwarding is not, and it is this boundary condition that most often terminates a triad.

A self-verifying generator. Because a single mis-scheduled μop produces a silently incorrect result and every hardware trial risks hanging the machine, we do not author triads by hand where we can avoid it. For each primitive a generator emits an ordered op-list over real register names, simulates that op-list against a model of the register file and memory that incorporates the addressing constraints of Section 1.4, and asserts the result against a reference implementation over many random inputs and against published known-answer vectors. Only thereafter does a packer lower the op-list to triads and the generator emit the C array. The packer is a single greedy left-to-right pass that closes the current triad whenever it fills its three slots, whenever a second memory operation would enter it, or whenever an operation would consume a register loaded earlier in the same triad, so that it mechanically enforces the two packing rules above, and it isolates control-transfer triads where required by Section 1.3. This discipline moves all correctness debugging into software, where it is immediate and safe, and converts the hardware-specific scheduling rules into a property of the toolchain rather than a burden on the programmer. The same procedure of generation, simulation, verification and packing underlies the Keccak permutation of Section 2; the small, densely irregular field kernels of Section 3 are instead hand-scheduled, but held to the identical verification discipline.

1.3 Patch RAM Budget

The writable patch RAM on Goldmont holds 128 triads, occupying the region `U7c00`–`U7dfc` in the MSRAM, in which each triad occupies four address units and must begin on an even address, and within which a small sub-range is reserved as a staging area and is not available to a primitive. This ceiling of 128 triads is the binding resource of the entire approach, and it shapes the design at the level of the algorithm rather than merely at the level of the code.

The budget selects the strategy. Because a patch is small, the central question for each primitive is how to accommodate a complete operation, including any iteration it involves, within 128 triads while keeping its working set resident. The answer is primitive-specific but is in every case a budgetary decision. An operation whose natural straight-line form would overflow the cap is restructured so that a single resident body is reused: an iterated computation is looped in place by means of a backward branch in the sequence word, with loop counters and constant tables held in the operand buffer rather than encoded in the program, so that the triad count becomes independent of the iteration count. This is the route taken by Keccak in Section 2, whose 24 rounds are looped over one resident round body. Where looping is too costly relative to the work performed per iteration, we instead fold a fixed and small number of iterations into one firing, as in the processing of two reduction words per `vmwrite` in the word-by-word Montgomery primitives, and re-enter from the native side, thereby trading a little additional input and output for remaining within the cap. By contrast, a primitive whose straight-line form already fits, as the Curve25519 field operations of Section 3 do at 42 and 66 triads, is bound not by the cap but by how densely

its arithmetic can be packed. In every case it is the relation between the work and the cap, rather than the instruction mix, that fixes the macro-structure of the implementation.

Spend the budget on residency, not on movement. Temporary registers do not survive across firings, so any value that must outlive a hooked instruction has to be carried in an architectural register or spilled to memory. This makes a single firing per operation the default arrangement, in which the inputs are loaded once in a prologue, the operation is computed entirely in registers, and the outputs are stored once in an epilogue, so that the memory traffic is paid once rather than once per iteration. The doubled register file is what renders this feasible, and we regard every triad spent on address arithmetic or data movement as a triad taken from the primitive itself. A concrete illustration arises in our own work, in which a variant that rebuilt an addressing base on each iteration in order to free one register measured strictly slower than a variant that kept the base pinned and instead borrowed a normally reserved register as scratch, because the cost of the rebuild exceeded the cost of the spill it removed. Under a fixed triad budget, the minimisation of data movement is the optimisation that matters most.

1.4 Findings

Using microcode as a target required us first to establish the undocumented behaviour of this particular Goldmont part, namely which μops exist and what they compute, how operands and flags are encoded, and which combinations are illegal. Each finding reported below was isolated by a dedicated probe and is now encoded as a rule in the generator and packer, so that a software-verified op-list lowers to a patch that runs correctly on its first hardware trial. We group the findings under the headings of the μop repertoire, the execution and flag semantics, the addressing, the control flow, and the performance.

Micro-op repertoire. The vertically encoded μops expose a RISC-like set that is sufficient for field arithmetic, comprising 64-bit `ADD`, `XOR`, `SHL` and `SHR`, the rotation `ROL`, which is a correct single μop for every rotation amount, the bitwise operation $\texttt{NOTAND}(d, a, b) = (\neg a) \wedge b$, register moves by means of `ZEROEXT`, and a $64 \times 64 \to 128$ multiplication $\texttt{MUL\_DSZ64\_DRR}(hi, a, b)$ that preserves a, returns the low half in b, and places the high half in hi. Immediate operand fields are 16 bits wide, so a 64-bit constant must be materialised by composition, using `ZEROEXT` followed by `SHL` and `XOR`, rather than in a single operation. Two pitfalls cost us considerable time. The first is that the memory operand forms, such as `ZEROEXT_DSZ64_DM`, do not perform general addressing but instead read a field of the triggering macroinstruction, so that general memory access is available only through the dedicated load and store operations. The second is that operations with an implicit memory operand cannot be used inside a patch at all. We therefore restrict the generator to register-only arithmetic together with the explicit load and store operations.

Execution and flag semantics. Intra-triad register dataflow is fully sequential, as described in Section 1.2, with the single exception that the result of a load does not forward within a triad; store-to-load forwarding across triads within a patch does function, however, which is what makes the spilling and reloading of scratch values safe. A 64-bit multiplication may occupy any slot of a triad, an earlier belief in a slot-zero restriction having not survived testing. The flags are the subtlest aspect of the machine, and we found two distinct flag domains. The first is a per-temporary-register condition that is set by an `ADD` and consumed by a conditional `SETCC`, and that persists until the next `ADD` to that register. The second is the architectural `RFLAGS`, which is frozen at patch entry and which is what `ADC` and the conditional branch read. The bridge from the first domain to the

second is unreliable in large patches. Three practical consequences follow and are encoded in the generator. All multi-word carry chains are built from per-register `ADD` and `SETCC` pairs rather than from `ADC`; `SETCC` writes correctly only to a temporary register and never to an architectural one, so a carry must be materialised in a temporary and then moved out; and a temporary needed across iterations must never be clobbered by an intervening carry computation. The first of these is precisely what enables the parallel carry chains of Section 3, and its boundary is precisely what limits them on the saturated representation discussed there.

Addressing. Loads and stores function only with a specific data-segment selector on this part, and a selector copied from unrelated code hung the core unrecoverably, so the working value is machine-specific and is fixed once. The immediate displacement is an 8-bit signed byte offset spanning ± 128 bytes, and an early layout that placed scratch beyond that range wrapped silently and corrupted state, a fault at first misdiagnosed as a dependency error. We therefore centre the operand buffer beneath its base pointer so that all displacements remain in range, and we use the register-indexed addressing form, which carries no such limit, for the tables and counters that lie beyond the reach of the immediate. As noted above, at most one memory operation may occupy a triad.

Control flow. A backward conditional branch encoded in the sequence word transfers control to an arbitrary triad and reads the architectural flags, ignoring any register operand encoded alongside it, and the arithmetic operation that sets the tested flag must occupy the same triad as the branch. This is the mechanism that underlies the in-place looping of Section 1.3, and Section 2 depends on it directly. Because temporary registers do not persist across firings, any state that must survive between firings has to reside in an architectural register or in memory, which is precisely why a complete, iterated operation is kept within a single firing wherever the triad budget permits.

Performance. The layer has two regimes of per-triad cost, and conflating them is the easiest way to misjudge a projected result. Measured as throughput, across many independent firings that the out-of-order front end is able to overlap, a triad costs approximately 0.49 cycles. A single dependent operation, by contrast, pays the latency, and in the absence of any overlap between firings and in the presence of a per-iteration branch the effective cost rises to roughly one triad per cycle. A primitive is therefore fast in this layer to the extent that its triad count is low and its critical path short, rather than because individual operations are cheap. The improvements we report across the primitives, which match or exceed both GCC at `-O3` and a state-of-the-art superoptimiser on most operations, derive from this structural source: the doubled register file removes the spills that dominate compiled code on an in-order core, and the three-wide hazard-aware packing shortens dependent chains. The advantage is structural, deriving from the elimination of data movement that the instruction set architecture cannot avoid, rather than from a higher peak throughput; and because the 128-triad cap forecloses extensive unrolling, the margin is modest on kernels that are already tight and latency-bound and large on kernels that are spill-heavy. The two worked examples that follow are chosen to exhibit both ends of this range.

2 Keccak-$f[1600]$ in Microcode

The first worked example instantiates the methodology of Section 1 on the Keccak-$f[1600]$ permutation. Keccak is the most demanding test of the layer in this work, because in contrast to the straight-line field-arithmetic primitives it is a 24-round loop with a per-round constant table, and it therefore exercises in-place looping, indexed addressing, and conditional control flow in addition to dense arithmetic packing. It is also the example in which the structural advantage of microcode is most readily stated, and, as we shall see, the example in which the limitations of that advantage are most clearly exposed.

The structural advantage. A Keccak state consists of 25 lanes of 64 bits. Native x86-64 exposes approximately 14 usable general-purpose registers, with the consequence that hand-written assembly cannot hold the entire state in registers and spills to the first-level cache continuously, and the fastest scalar implementations recover residency only by fully unrolling all 24 rounds. The micro-architectural register file is 32 entries wide, comprising 16 general-purpose and 16 temporary registers, which is sufficient to hold the complete 25-lane state resident not merely within a single round but across all 24. Our design rests entirely on this property, in that the 25 lanes are loaded once, the 24 rounds are evaluated with the whole state resident in registers, and the lanes are stored once at the end. We achieve residency without unrolling, which the 128-triad capacity of Section 1.3 would not in any case permit, and we expend the resulting register headroom on per-round transformations that register-starved native code is unable to perform.

2.1 State Layout and Residency

Register allocation. The 25 lanes, indexed by $i = 5y+x$, are assigned to a fixed set of 25 register names, namely the 13 general-purpose registers `RDI`, `RSI`, `RBX`, `RDX`, `RBP`, `R8` ... `R15` followed by the 12 temporaries `TMP0` ... `TMP11`. The register `RCX` holds the buffer base and is never repurposed, while the remaining five registers, namely `RAX` and `TMP12` ... `TMP15`, serve as scratch and hold the five resident D values of θ during a round and the temporaries of χ once those values are no longer needed. The file is one register short of what full D-residency requires, and we therefore borrow `RSP` as a 32nd data register, saving it to the buffer in the prologue and restoring it in the epilogue, having verified in Section 1.4 that this is safe because the patch never touches the stack. This single substitution is what allows D to live in registers rather than being re-read from memory once per lane.

Buffer organisation. The operation is backed by a single contiguous array of 56 lanes, and the base pointer is centered at `RCX = &buf[16]` so that every displacement fits within the 8-bit signed offset field discussed in Section 1.4, as summarised in Table 1. Only the 25 state lanes are accessed by the prologue and epilogue. The column-parity scratch is spilled and re-read within a round, whereas the round-constant table lies beyond the reach of the immediate offset and is therefore addressed through the register-indexed load form. The loop counter is stored not as a round number but as the byte offset of the current constant, so that the constant fetch requires no scaling arithmetic on its critical path.

2.2 The Looped Round Body

The entire permutation is evaluated within a single hooked firing, comprising a prologue that loads the 25 lanes using one single-load triad per lane, since at most one memory operation may share a

Table 1: Operand buffer of 56 lanes, with the base register centered at `RCX=&buf[16]`.

lane	offset(`RCX`)	name	contents
0–24	$-128 \ldots +64$	state	resident in 13 GPR + 12 TMP during a round
25–29	$+72 \ldots +104$	DSCR	θ-C parity scratch (spilled, re-read)
30	+112	COUNTER	RC byte-index, initialised to 128 and advanced by 8 each round
31	+120	RSPSAVE	saved `RSP` (borrowed as a data register)
32–55	(indexed)	RC table	24 round constants, addressed only by register index

triad, a round body that is looped 24 times by means of a backward branch, and an epilogue that stores the lanes back to memory. We describe the body in order of execution, giving particular attention to the two stages at which the layer confers its advantage.

The θ stage and the residency of D. The column parity $C_x = \bigoplus_y A[x+5y]$ stands at the head of every round's dependency chain, and we therefore shorten its latency. In place of the reference implementation's depth-four left-associated chain we emit a balanced tree of depth three, using `RSP` as a second accumulator so that the two halves are computed in parallel, and the packer places the dependent exclusive-or operations in successive slots of a single triad, as shown in Listing 1. The five values C_x are then spilled to the buffer, because the registers are required immediately afterwards, and the mixing lanes $D_x = C_{x-1} \oplus \mathrm{rol}(C_{x+1}, 1)$ are computed into the five scratch registers and kept resident for the remainder of the round. This use of the resident D values is the principal benefit of the borrowed `RSP`, and it eliminates the per-lane reloads of D, amounting to 25 additional memory triads per round, that earlier versions of the kernel incurred.

Listing 1: The depth-three parity tree of θ, in which a single column's tree is packed into the slots of one triad while the spill of the previous column is interleaved.

```
{ XOR(RAX,RDI,R8),   XOR(RSP,R13,TMP2),  XOR(RAX,RAX,RSP),  NOP_SEQW }
{ XOR(RAX,RAX,TMP7), STAD(RAX,RCX,72),   XOR(RAX,RSI,R9),   NOP_SEQW }
```

The fused θ-apply, ρ, and π stage. The natural concern when looping a round is that π relabels lane positions, so that the lane held in a given register would appear to drift from one round to the next. This concern is unfounded, and the reason is structural. The permutation π on the 25 lanes consists of one fixed point, namely lane 0, together with a single 24-cycle. By applying π in place along that cycle, which is to say by walking the cycle and moving each element into its destination register while folding in the exclusive-or of θ and the rotation of ρ, every lane is returned to its canonical register at the end of the round. Round r+1 consequently observes exactly the register layout that round r observed, so that a single identical body may be looped 24 times with no per-round renaming and no two-body alternation scheme. The fusion costs one exclusive-or and one rotation per lane, and D is read directly from registers without any memory access.

The χ stage without auxiliary moves. The step χ computes, for each row, $A_x \oplus= (\neg A_{x+1}) \wedge A_{x+2}$, which corresponds exactly to the `NOTAND` primitive. If this were performed naively in place, the overwriting of A_x would destroy a value that the subsequent lanes still require, which forces two auxiliary moves per row, or ten per round, in register-starved native code. Because the five D registers are now dead, we have the headroom to compute all five `NOTAND` terms of a row first, placing them in scratch, and then to apply the five in-place exclusive-or operations, as shown in

Listing 2, so that no auxiliary moves are required at all. This was the single largest measured improvement, as reported in Section 2.5, and it is a direct consequence of residency, in that it is precisely the transformation that native code is unable to make.

Listing 2: The move-free χ stage, in which the five `NOTAND` terms of a row are computed into the now-dead D-registers before the five in-place exclusive-or operations are applied.

```
{ NOTAND(RAX,RSI,RBX), NOTAND(TMP12,RBX,RDX), NOTAND(TMP13,RDX,RBP), NOP_SEQW }
{ NOTAND(TMP14,RBP,RDI),NOTAND(TMP15,RDI,RSI),XOR(RDI,RDI,RAX),      NOP_SEQW }
{ XOR(RSI,RSI,TMP12),   XOR(RBX,RBX,TMP13),   XOR(RDX,RDX,TMP14),    NOP_SEQW }
```

The ι stage and loop control. Because the counter lane stores the byte offset of the current round constant directly, the step ι reduces to a load of the index, an indexed load of the constant, and a single exclusive-or into lane 0, with no scaling arithmetic on the path from ι to lane 0. The counter is then advanced by 8 and persisted in the spare slots of the same triad, after which a single forced triad performs the loop test and the backward branch. In that triad an exclusive-or of the index against the end marker sets the architectural zero flag, and a conditional branch within the same triad returns control to the top of the body for as long as the index differs from 320, which lies just past the last constant, thereby producing exactly 24 iterations, as shown in Listing 3. We count upward rather than downward because decrement-style operations did not behave correctly on this part, as recorded in Section 1.4.

Listing 3: Loop control, in which the flag-setting exclusive-or and the backward conditional branch must occupy the same triad.

```
{ XOR(TMP12,TMP14,320), NOP, UJMPCC_CONDNZ(TMP12, 0x7c68 /*loop_top*/), NOP_SEQW }
```

2.3 Triad Budget

The complete permutation compiles to 108 triads, which lies comfortably within the 128-triad patch RAM. These comprise a 26-triad prologue, consisting of one `RSP` save and 25 single-lane loads, a 56-triad body that includes the constant lookup and the loop test, and a 26-triad epilogue, consisting of 25 stores and the `RSP` restore. The body begins at `U7c68`, which is the branch target. Because the body is looped rather than unrolled, the static footprint is independent of the number of rounds, whereas the dynamic count, that is, the number of triads actually executed per permutation, is $26 + 24 \times 56 + 26 = 1396$. The 20 triads of headroom beneath the cap constitute genuine slack, but, as Section 2.5 explains, they cannot be spent on unrolling for additional parallelism, because even a single additional round body would not fit alongside the existing one.

2.4 Verification

In accordance with Section 1.2, no triad is written by hand. The generator emits the round as an op-list, executes it on a register-and-memory simulator that models the 8-bit signed-offset limit, so that an out-of-range displacement manifests as an assertion failure in software rather than as a silent corruption of state on hardware, and asserts equality with a reference Keccak over 64 random states across all 24 rounds before any triads are emitted. On hardware the patch is then checked in two ways, first lane by lane against the same reference, and second, more stringently, against the published $f(0)$ and $f(f(0))$ known-answer vectors of the Keccak team, so that a correlated error present in both the reference and the patch cannot escape detection. The all-zero anchor reproduces the canonical value `lane0` $=$ `0xF1258F7940E1DDE7`.

Table 2: Best result for each contender over 24 compiler-and-optimisation configurations, measured base-pinned and same-process, where a smaller value is better.

contender	min (cyc/perm)	median	best config
microcode (looped)	**1910**	1916	gcc-13 -Os
`opt64lcu24`	2045	2054	gcc-11 -Os
`x86_64_asm`	2061	2069	clang-14 -O2
`opt64lcu24shld`	10053	10064	clang-17 -Os
`x86_64_shld`	10086	10097	gcc-13 -O2

2.5 Performance

Measurement methodology. Timing on this part is subject to a pitfall that invalidates naive comparisons. The `rdtsc` instruction counts at a fixed reference rate of approximately 1.1 GHz, whereas the core bursts to approximately 2.4 GHz under load, with the consequence that the same work is reported as roughly 2.2 times more or fewer ticks depending on the power state during measurement. Comparing a fresh measurement against a baseline captured in a separate run is therefore meaningless. We instead time the microcode and all of the scalar contenders back to back within a single process, after a warm-up phase has driven the core to a steady frequency, so that both observe the same clock and the ratio between them is frequency-invariant; for absolute cycle counts we additionally pin the core to its base frequency so that the reported ticks approximate true cycles. We compare against the genuinely fastest scalar baselines from SUPERCOP, swept over 24 compiler-and-optimisation configurations with the best result selected for each contender, in the same manner as the SUPERCOP autotuner.

Result. Under base-frequency, same-process measurement the looped permutation completes in 1910 cycles per permutation, as against 2045 cycles for the fastest scalar contender, `opt64lcu24`, and 2061 cycles for the hand-tuned `x86_64_asm`, as reported in Table 2. This corresponds to a ratio of 0.934, an improvement of approximately 7%, and the microcode is the fastest of all contenders. The ratio is the robust quantity, since the same harness measured at burst frequency reports approximately 874 cycles against 942, which is the identical ratio of 0.93 scaled by the 2.4/1.1 power-state factor.

The source and the bound of the improvement. The margin is modest by design, and the latency model of Section 1.4 explains why. A single isolated firing sustains 0.49 cycles per triad, but this is a throughput figure obtained by overlapping independent firings, whereas the looped permutation is a strict dependency chain, in which round N requires round $N-1$, with a backward branch on each round and no overlap between firings, so that it pays the true latency of approximately 1.37 cycles per triad and the 1396 dynamic triads accordingly cost approximately 1910 cycles. The microcode is therefore latency-bound at roughly one triad per cycle, whereas the hand-tuned scalar assembly extracts approximately three operations per cycle from the rotate and `ANDN` instruction-level parallelism of Goldmont. We cannot close this gap with additional parallelism, because the chain $\theta \to \rho \to \pi \to \chi$ is inherently sequential and the 128-triad cap precludes the unrolling that would expose independent work. The entire advantage is consequently the per-round work that native code cannot perform, namely full register residency, with the state input and output paid once per permutation rather than once per round, the borrowed 32nd register that keeps D resident, and the move-free χ stage. Table 3 shows the improvement assembled from these structural contributions, each measured back to back.

Table 3: The improvement assembled from successive structural optimisations, expressed as the ratio to the naive residency-only baseline.

step	change	ratio
0	per-lane D in buffer, with a naive χ using auxiliary moves	1.00×
1	D resident in registers via the borrowed `RSP`, with no base rebuild	0.99×
2	move-free χ using the dead D-registers as scratch	0.95×
3	depth-three θ tree and byte-indexed RC, shortening the critical paths	0.93×

In summary, a complete and known-answer-verified 24-round Keccak-$f[1600]$ runs entirely in Goldmont microcode, looped within a single `vmwrite` firing, in 108 triads, and outperforms the fastest hand-tuned scalar assembly on this microarchitecture by approximately 7%. The result is a clear demonstration of the character of the layer established in Section 1, in that its advantage is structural, deriving from the elimination of the spills and data movement that the instruction set architecture cannot avoid, rather than from a higher peak throughput, and it is greatest precisely where native code is most constrained by the shortage of registers.

3 Curve25519 in Microcode

The second worked example instantiates the methodology of Section 1 on the field arithmetic that underlies X25519. Whereas Keccak (Section 2) was realised as a single large, control-flow-heavy patch looped in place, a Curve25519 field operation has the opposite character. It is a small, straight-line, multiplication-dominated kernel that is invoked many hundreds of times within a single scalar multiplication. The primitive that we encode in microcode is consequently the field multiplication and squaring modulo $2^{255} - 19$, and the question we address is whether a microcode patch can evaluate these operations more quickly than the best available native code, namely compiler output, an automatically generated and formally verified backend, a superoptimiser, and expert hand-written assembly.

The structural advantage. In this setting the advantage of the layer is not register residency, since five limbs occupy only a small fraction of the register file. The performance-critical path of a reduced multiplication is a sum of partial products that is accumulated into a small number of running totals and then propagated as carries, and on native x86 the limiting factor is the carry rather than the arithmetic. Because the architecture exposes a single carry flag, multi-word carry propagation is forced to serialise through the `ADC` instruction, and a two-ALU in-order core cannot overlap that dependent chain with the multiplications that feed it. The microcode layer addresses this limitation in two ways, both of which follow from the properties established in Section 1.4. A single triad fuses a multiplication, an accumulation, and a carry capture into one issue bundle, and the per-register `SETCC` flag domains allow several carry chains to progress concurrently without contending for one architectural flag. This concurrency of carry propagation is the field-arithmetic counterpart of the register residency that Keccak exploited, in that it is work the instruction set architecture is unable to express.

3.1 Representation and Operations

We adopt the unsaturated radix-2^{51} representation, in which a field element is stored as five 64-bit limbs that each carry 51 significant bits. This is the representation emitted by fiat-crypto and used by the Bernstein–Schwabe `amd64-51` implementation. Multiplication is the 5×5 schoolbook of 25 partial products, and the reduction implied by $2^{255} \equiv 19$ is folded directly into the cross terms so that no separate reduction pass is required, giving

$$c_i \;=\; \sum_{j \le i} a_{\,i-j}\, b_j \;+\; 19 \sum_{j > i} a_{\,5+i-j}\, b_j .$$

Squaring exploits the symmetry $a_i b_j = a_j a_i$ to reduce the 25 products to 15, and the doublings $2a_i$ and folds $19a_i$ that this requires are precomputed by the caller and supplied in registers. The carries that arise during accumulation are short, because each limb is reduced to 51 bits and the remaining 13-bit overflow, which together with the limb width fills a 64-bit word, is shifted into the following limb. The brevity and independence of these carries are what make the representation well suited to the layer, a point to which we return in Section 3.5, since it is exactly where the 4×64 saturated representation behaves differently.

3.2 The Patch: Fused Multiply–Accumulate and Parallel Carry Chains

A complete field multiplication or squaring is carried out by a single execution of one hooked macro-instruction, an event we term a firing. The native ladder triggers the operation with that one

instruction, whereupon the corresponding patch evaluates the entire reduced product in registers, which is to say all of the partial products together with the carry propagation and the reduction modulo $2^{255} - 19$, and returns control to the native side with the result in place. There is in consequence neither any per-limb looping nor any repeated round-trip between native code and microcode within a single field operation. The two patches reside in the patch RAM simultaneously, the multiplication occupying 66 triads from `U7c00` and the squaring 42 triads from `U7d08`, and they are therefore reached through two distinct match registers that redirect two different macro-instructions, in the manner described in Section 1.2. The multiplication is hooked to `vmwrite`, whose opcode is `0x0cd8`, while the squaring is hooked to `vmread`, whose opcode is `0x0618`, so that the native code fires the multiplication by issuing a `vmwrite` and the squaring by issuing a `vmread`. Using a separate entry instruction for each patch is what allows both to be installed at once, since a single hooked instruction can be redirected to only one entry point. The caller passes the five limbs, together with the precomputed $19b_j$ folds in the case of multiplication, in fixed registers. A short preparation stage then materialises the remaining folds in place, computing $19b_1, \ldots, 19b_4$ with three multiplications packed into one triad by means of the slot-write rule described in Section 1.4, after which the kernel traverses the limbs and emits for each of them a chain of fused multiply–accumulate triads of the form shown in Listing 4.

Listing 4: The fused multiply–accumulate–carry core of the squaring kernel for a single limb. The register `TMP0` holds the low accumulator, `R8` the high accumulator, `TMP9` the running carry sum, and `TMP15` the transient `SETCC` carry. A multiplication, an accumulation, and a carry capture share one triad.

```
{ ADD(TMP0,TMP0,RDI), SETCC(TMP15,TMP0), ADD(R8,R8,RCX),  NOP_SEQW } // acc lo; carry; acc hi
{ ZEROEXT(TMP9,TMP15),MUL(RCX,RBX,R13),  ADD(TMP0,TMP0,R13),NOP_SEQW } // next product while
    accumulating
```

Three concurrent accumulators. The decisive feature of the kernel is that the sum of partial products is maintained as three independent chains, none of which touches the single architectural flag. The low halves of the products accumulate into `TMP0`, the high halves accumulate into `R8`, and the carries produced by the individual accumulations are summed into `TMP9`. Each chain captures its own carry by means of a `SETCC` into a per-register flag domain, as established in Section 1.4, with the consequence that the carry capture occupies a spare slot of the multiplication's triad rather than serialising behind it. The transition between limbs is handled by a single `OR` that seeds the next limb's low accumulator from the two shifted pieces of the previous limb's carry, namely the high part of the accumulator shifted right by 51 bits combined with the carry sum shifted left by 13 bits. A native multiplication bound by `ADC` cannot reproduce this scheme, because it would be obliged to thread every carry through one flag.

Authoring and verification. In contrast to the Keccak permutation, the Curve25519 field patches are hand-scheduled rather than generator-emitted. The kernels are small enough, at a few tens of triads, to lay out by hand, and the dense and irregular packing of multiplications and carries leaves little for a generic packer to improve. They are nevertheless held to the discipline described in Section 1.2. Each patch is verified against the fiat-crypto reference, which is also the backend that the superoptimiser targets, over a large number of random inputs, and the assembled X25519 is checked against the known-answer vectors of RFC 7748 before any performance claim is made.

3.3 Triad Budget

The squaring kernel compiles to 42 triads and the multiplication kernel to 66 triads, and both therefore lie comfortably within the 128-triad capacity of the patch RAM. This is the qualitative respect in which Curve25519 differs from Keccak, because the patch budget is not the binding constraint and the governing design variable is not how to make the operation fit but how densely it can be packed. Every multiply–accumulate–carry sequence that is folded into a single triad shortens both the patch and its critical path, and the optimisation history of these kernels consists largely of such merges. The multiplication, for example, was reduced from 73 to 66 triads solely by packing the preparation stage's multiplications three to a triad and by drawing the per-product carry capture into the free slots of the multiply–accumulate triads.

3.4 Integration into X25519

A field multiplication or squaring is invoked several hundred times in the course of a single scalar multiplication, which comprises a 255-step Montgomery ladder with several field operations per step followed by a Fermat inversion that is itself dominated by squarings. Because each invocation is a single hooked firing, a `vmwrite` for a multiplication and a `vmread` for a squaring, the per-firing dispatch overhead of approximately four cycles established in Section 2 is amortised against the cost of the field operation.

Our complete implementation, which is the fastest that we constructed and which we denote the inlined ladder, realises the entire scalar multiplication as inline assembly built around these two patches rather than as C that calls them through function wrappers. The fifteen field elements that a ladder step manipulates are held together in a single stack-resident state structure at fixed offsets, and the assembly addresses them through a base register that is pinned to that structure for the duration of the step. The microcode firings are emitted inline within one large assembly block per ladder step, with the consequence that the compiler spills registers only around the block as a whole rather than at every field operation. Within that block, consecutive operations are chained through registers wherever the dataflow permits, in that the output limbs of one operation are passed directly as the inputs of the next and the reload that would otherwise begin the second operation is omitted; a single ladder step contains five such chains, of which one example is the addition forming $A = x_2 + z_2$ feeding directly into the squaring $AA = A^2$. The Fermat inversion is realised in the same manner, as one inlined assembly block in which the long sequence of squarings is chained through registers, so that only the first squaring loads from memory and only the last stores its result. This arrangement extends the principle of Section 1.3, namely that data movement is the quantity to be minimised, from the interior of a single field operation to the boundaries between successive operations, and it is the reason that the inlined ladder is faster than the otherwise identical implementation that invokes the field operations through function calls.

In order to measure the contribution of the field operation in isolation, which the inlined ladder by its construction entangles with the integration, we additionally conduct a controlled comparison in which the ladder, the inversion chain, and the conditional swap are held fixed and only the field backend is varied. This same-ladder comparison is conducted against four alternatives, namely hand-written `__uint128_t` C, the automatically generated C produced by fiat-crypto, the Goldmont-tuned assembly produced by the CryptOpt superoptimiser, and the hand-written `amd64-51` assembly of Bernstein and Schwabe. We perform the comparison both within our own ladder and within the `amd64-51` ladder, so that the microcode backend is evaluated against expert assembly on the framework of the assembly's own authors.

Table 4: Same-ladder field-operation comparison, reported as the geometric mean of the per-configuration minimum-cycle ratios for a complete X25519, where a smaller value indicates faster execution and the field backend is the only component that varies within each block.

ladder	field-operation backend	ratio vs. microcode
ours	hand-written `__uint128_t` C	1.24×
ours	fiat-crypto (autogenerated C)	1.20×
ours	CryptOpt (superoptimised asm)	1.22×
ours	**microcode**	1.00×
amd64-51	Bernstein–Schwabe hand asm	1.07×
amd64-51	**microcode**	1.00×

3.5 Performance

Methodology. We follow the measurement discipline of Section 2, timing at the base frequency within a single process and sweeping over 24 compiler-and-optimisation configurations while retaining the best result for each contender, in the same manner as the SUPERCOP autotuner. Since the comparison is conducted at the level of a complete X25519, the headline figure is the geometric mean of the per-configuration ratios, which collapses the grid to a single number for each backend.

The field-operation result. When the ladder is held fixed, the microcode field operations are the fastest backend that we measured. Within our own ladder the microcode is faster than hand-written C by a factor of 1.24, faster than fiat-crypto by a factor of 1.20, and faster than the CryptOpt superoptimiser by a factor of 1.22, each expressed as a geometric mean over the 24 configurations and reported in Table 4. The microcode therefore outperforms optimised C, a formally verified automatically generated backend, and a state-of-the-art superoptimiser, with every component other than the field operation held constant. When the microcode is substituted for the hand-written assembly in the `amd64-51` framework of Bernstein and Schwabe, it remains faster by a factor of 1.07, so that the advantage persists even against expert hand-tuned x86 assembly. The aggregate improvement is driven by the squaring, which in isolation runs in 37 cycles and is therefore 26% faster than fiat, while the multiplication runs in 58 cycles and is 3% faster; since a ladder is dominated by squarings, the squaring advantage governs the overall result.

The integration result. The inlined ladder of Section 3.4 is the fastest of our implementations, completing X25519 in 300,197 cycles, as against 304,897 cycles for the otherwise identical implementation that invokes the same microcode field operations through function calls, as reported in Table 5. The register chaining and the inlining of the firings therefore account for the difference of approximately 1.5% between the two, a margin consistent with the expectation that store-to-load forwarding hides most, though not all, of the inter-operation memory traffic.

The limits of the result. Considered end to end, however, the fastest X25519 on Goldmont is not our own. That position is held by the Bernstein–Schwabe `amd64-64` implementation, which uses the 4×64 saturated representation with radix 2^{64} and completes in 271k cycles, ahead of our fastest implementation, the inlined microcode ladder, at approximately 300k cycles, as reported in Table 5. A more troubling observation for the layer is that when we port the microcode field operations to that same 4×64 representation they regress by roughly a factor of two relative to the native assembly. The explanation lies precisely in the carry structure discussed in Section 3.1, because a saturated 4×64 multiplication requires long, full-width carry chains across 64-bit limbs,

Table 5: End-to-end X25519 performance, given as the best result for each contender over 24 configurations in minimum cycles. The field-operation advantage does not overcome the representation-level advantage of `amd64-64`, and the microcode 4×64 port is where the layer's single-flag carry model is exposed.

contender	representation and backend	min (cyc)
amd64-64 / asm	4×64 saturated, hand asm	271 314
ours / ucode-inline	5×51, microcode (inlined)	300 197
ours / ucode	5×51, microcode	304 897
amd64-51 / ucode	5×51, microcode (B–S ladder)	332 323
donna_c64	5×51, portable C	335 736
ours / fiat	5×51, fiat-crypto C	354 874
amd64-51 / asm	5×51, hand asm	355 975
ours / cryptopt	5×51, CryptOpt asm	379 941
amd64-64 / ucode	4×64 saturated, microcode	510 750

which is the pattern that our flag model handles least well. Since `ADC` reads the architectural flag domain that is frozen at patch entry, as noted in Section 1.4, a four-deep saturated carry must be emulated with serial `SETCC` operations, and the advantage conferred by parallel carry propagation is thereby lost. The unsaturated 5×51 representation is favourable to the microcode for the converse reason, namely that its 13-bit carries remain short and mutually independent. The representation that is most favourable to native assembly is consequently the one at which the layer is weakest, and the reverse holds as well.

Summary. At the level at which the paper makes its claim, namely the field operation with the surrounding ladder, inversion, and conditional swap held fixed, the microcode is the fastest Curve25519 field backend that we measured on Goldmont, surpassing optimised C, fiat-crypto, the CryptOpt superoptimiser, and expert hand-written assembly. The result has the same character as that of Keccak in Section 2, in that it derives from a structural per-operation advantage that the instruction set architecture cannot reach, which in this instance is the fused multiply–accumulate together with the parallel `SETCC` carry chains rather than register residency. As with Keccak, the boundary of the result is stated honestly. The end-to-end advantage belongs to a different representation, the 4×64 saturated form, and that representation is exactly where the one structural weakness of the layer, namely long carry chains under a single-flag emulation model, becomes apparent.